

INFO-F-307 – Génie logiciel et gestion de projets
Organisation du cours, des séances d'exercices et du projet.

Professeur : F. Pluquet

Assistants : D. Alnakeb, H. Callebaut, A. Cnudde, M. Colot, A. Laval et Y. Molinghen

Table des matières

1	Introduction	2
2	Organisation et conditions du travail	2
3	Livrables	4
4	Contact	5
5	Dates importantes	5
6	Critères d'évaluation	6
6.1	Outils	7
7	Objectifs du projet	8
8	La fin du projet	8
A	Présentation et contenu des histoires	9

1 Introduction

Le cours de *Génie logiciel et gestion de projets* sera principalement consacré à l'élaboration d'un projet. Trois séances de travaux pratiques seront dispensées par les assistants (voir calendrier ci-après). Les matières que recouvriront ces séances sont :

- Introduction générale
- **Git** et les branches
- Le test driven development (TDD)
- Le refactoring

En ce qui concerne le projet, le projet sera implémenté en langage Java et l'environnement de développement qui sera utilisé est **IntelliJ**.

Les autres séances de travaux pratiques seront entièrement consacrées à l'avancement du projet. Elles serviront de plage horaire commune à tous les membres d'un groupe, vous permettrez d'organiser votre travail et nous permettront de suivre l'avancement de vos projets et de répondre à vos éventuelles questions.

2 Organisation et conditions du travail

Nous avons décidé d'embaucher les bénévoles que vous êtes pour réaliser un projet. Des *groupes* de 10 développeurs réaliseront le projet.

Programmation XP Nous allons travailler avec la méthodologie XP, avec des itérations de 2 semaines (excluant les congés, voir le calendrier ci-après). Pour chaque itération, un livrable précis sera produit. Il devra être déposé sur un dépôt Git en ligne. Les informations à ce sujet vous seront transmises prochainement.

Gestion de versions Les groupes doivent obligatoirement utiliser le système de contrôle de versions **Git** qui leur permet de gérer l'évolution du code source de leur application. Tout au long du projet, ce dépôt sera utilisé pour évaluer le travail et les progrès de chaque groupe d'étudiants.

Contraintes de temps Les livrables à rendre pour chaque itération doivent être déposés avant l'échéance imposée. Puisque l'un des objectifs du projet consiste à apprendre à respecter les échéances imposées lors d'un projet de développement logiciel, tout retard sera lourdement pénalisé. Il est à noter que si les livrables ne nous sont pas fournis à temps pour une itération, les cotes du groupe pour cette dernière seront **nulles** (sans recours possible). Si cela arrive une deuxième fois, la cote finale du groupe sera alors **nulle** (sans recours possible non plus).

Contraintes de présence Les présences aux séances organisées dans le cadre du projet sont obligatoires. Toute absence non justifiée sera sanctionnée sur la note finale.

Composition des groupes Nous vous laissons le soin de former vos groupes de 10 étudiants. Cette composition doit être communiquée au plus tard le **vendredi 13 février 2026** exclusivement via le Form suivant :



FIGURE 1 – Formulaire disponible sur <https://forms.office.com/e/nJWbjjHNtt>

Si vous ne parvenez pas à former un groupe, vous devez tout de même remplir ce formulaire avant la date limite. **Après cette date, les étudiants n'ayant pas de groupe et n'ayant pas complété le formulaire pour nous prévenir auront une note de 0/20 au projet.**

Pendant l'année académique, la composition des groupes est susceptible d'être changée à tout moment ! Les assistants peuvent échanger des membres de groupes ou en rajouter dans le but de restructurer les groupes.

En cas de problème dans votre groupe (quelqu'un ne vient jamais aux rendez-vous, ne travaille pas, ou autre), vous essayez d'abord de gérer cela en interne. La gestion d'un groupe peut prendre du temps et vous devez être proactifs pour cela. Si la situation ne peut être résolue en interne, vous pouvez contacter les assistants, qui prendront alors les mesures nécessaires pour la bonne continuation du projet et qui, si besoin est, avertiront le professeur (Monsieur Frédéric Pluquet), afin de prendre une décision définitive.

Exigences de qualité Les fonctionnalités, la complétude et la qualité du code produit et des tests unitaires peuvent être évaluées par les enseignants après l'échéance imposée pour chaque étape-clé. La couverture du code par des tests unitaires pourra également être évaluée.

Travail en groupe Tous les étudiants d'un groupe travailleront **collectivement** pour raffiner les *User Stories* afin de réaliser un projet cohérent. Chaque membre du groupe travaillera sur la conception et l'implémentation en Java de l'application. Chaque étudiant devra travailler avec ses **propres identifiants Git** afin d'évaluer la part personnelle de chacun si nécessaire. **La contribution de chaque étudiant pourra être évaluée via l'historique du dépôt Git**. Si un étudiant ne démontre pas une contribution significative au projet (peu ou pas de commits, pas de travail visible sur le dépôt, sur la branche principale), il pourra être exclu du projet. L'étudiant exclu devra repasser le projet l'année suivante.

3 Livrables

Pour chaque itération, nous vous demanderons de nous fournir :

- **avant 23h59 le lundi clôturant l'itération**, tous les livrables définis plus bas sur le *repository* mis à votre disposition.
- **avant 23h59 le mercredi commençant l'itération**, les histoires de la nouvelle itération en tenant compte des remarques (s'il y en a) des assistants que vous auriez reçues.

Les exigences fonctionnelles et non-fonctionnelles de l'application seront établies grâce à une version préliminaire fournie des “User Stories”. Chaque groupe travaillera **collectivement** pour raffiner les *User Stories*. Chaque groupe, de manière individuelle, classera les user stories selon leur ordre d'importance (P1, P2 et P3) et les risques (R1, R2 et R3). L'ensemble des exigences devrait former un tout cohérent.

La conception de l'application peut être modélisée en utilisant le langage de modélisation UML ² mais aucune documentation UML ne vous est explicitement demandée durant le projet.

Chaque groupe doit travailler individuellement à l'implémentation. Le plagiat sera lourdement pénalisé et peut mener à un échec total pour le cours ou même pour l'année d'études (cf. règlement d'études ULB).

Les tests unitaires doivent être utilisés durant toute la phase d'implémentation. Nous imposons une approche de développement dirigé par les tests : l'écriture du code doit être alternée avec l'écriture des tests, et tous les tests doivent passer avant de progresser dans l'écriture du code. Une autre bonne pratique est l'utilisation de *regression testing*. Pour chaque erreur rencontrée (ou bug), un test unitaire doit être écrit qui met en évidence l'erreur. Vous n'aurez ainsi pas à comprendre et résoudre deux fois le même problème.

Nous exigeons un code et des tests de qualité pour l'application, comme cela est détaillé en section 6.

Livrables par itération : histoires, code source, tests unitaires et autres documents

Les éléments à rendre sont :

- Le code source Java. Cela inclut :
 - La documentation pour les programmeurs, au format Javadoc ¹. La Javadoc doit contenir une documentation complète pour le projet ; les packages ; les classes, interfaces et énumérations ; les méthodes principales ;
 - Les tests unitaires ;
- Un document qui décrit :
 - le suivi de planification du projet : a-t-on implémenté toutes les fonctionnalités, comment les difficultés techniques ont-elles été contournées ? ...
 - la motivation des choix pris lors de la conception : design patterns principaux, ...
- Un burndown chart actualisé ;
- Un fichier actualisé, au format XLS ou Markdown (.md), qui reprend la répartition des tâches développeur pour les itérations effectuées ;
- Les histoires actualisées en terme du contenu et estimation des points au format XLS

1. <http://java.sun.com/j2se/javadoc>

- ou Markdown (.md) ;
- Le fichier exécutable `.jar` contenant la version du code que vous présenterez pendant la démonstration.

Tous ces livrables devront être stockés dans la branche **master** du dépôt **Git** correspondant à votre groupe. Votre repository sera initialisé avec des documents déjà préremplis pour vous aider et uniformiser la structure utilisée. Merci de garder cette structure tout au long de votre projet.

4 Contact

Une fois les groupes formés, chaque groupe aura deux assistants attitrés. Vos assistants attitrés seront votre client et votre coach et ils vous indiqueront quel moyen est à utiliser pour le contacter pour toute question. Aucun retour ne peut être donné si vous utilisez un autre moyen que celui défini par vos assistants.

5 Dates importantes

Le projet est jalonné de dates reprises dans la table 1.

Sem.	Actions	Date
21	Premier TP	3 février
22	Deuxième TP	10 février
23	Pas de TP (Mardi gras)	17 février
24	Début Itération 1	24 février
25	Troisième TP	3 mars
26	Échéance It. 1 + Début Itération 2	10 mars
27	Discussion du code It. 1	17 mars
28	Pas de TP (Printemps des sciences)	24 mars
29	Échéance It. 2 + Début itération 3	31 mars
30	Discussion du code It. 2	7 avril
31	Échéance It. 3 + Début itération 4	14 avril
32	Pas de TP (Vacances de printemps)	21 avril
33	Pas de TP (Vacances de printemps)	28 avril
34	Discussion du code It. 3	5 mai
35	Fin de l'itération 4 (Pas de TP)	12 mai

TABLE 1 – Calendrier du projet

Entre deux itérations successives, deux étapes-clés sont prévues.

Échéance It. n et Début Itération $n + 1$ Avant le lundi qui précède à 23h59, vous nous fournissez les livrables pour l'itération n (comme définis plus haut). Durant le TP du mardi, vous faites une démonstration de maximum 10 minutes et nous discutons des histoires à réaliser pour l'itération suivante.

Discussion du code It. n Nous examinerons avec vous les détails des mises à jour du code source, du code des tests, de la vue d'ensemble de l'architecture et de tout autre document introduit lors de la dernière itération. Nous utiliserons le système d'issues sur votre repository `Git` pour y indiquer des commentaires quant à votre code et à votre architecture. Toutes les issues doivent être prises en considération et fermées (ou commentées si nécessaire) le plus tôt possible et de toute façon, avant la fin de la 4e itération. **Attention : ces commentaires ne seront absolument pas exhaustifs et ne traiteront que de certains éléments de vos livrables. D'autres commentaires seront donnés durant le cours théorique. A vous d'extraire l'information nécessaire quant aux corrections à apporter à votre code et à votre architecture.**

Il est important de noter que **3 séances du cours théoriques** seront utilisées pour analyser les erreurs les plus fréquentes retrouvées dans vos codes. Ces remarques expliquées durant le cours théorique ne se retrouveront peut-être pas dans vos commentaires de groupe (via les issues ou oralement par les assistants) mais seront considérées comme vues et nous en tiendrons compte lors de l'évaluation finale.

6 Critères d'évaluation

Votre travail sera évalué sur **la qualité de votre implémentation** (voir ci-après) mais également sur **la qualité de l'application de la méthodologie** expliquée au cours. Vos coachs prendront note chaque semaine de la qualité de votre approche méthodologique du projet. Ces différentes notes feront parties intégrantes de la note finale de votre projet.

Votre travail **d'implémentation** sera évalué selon les critères suivants :

Conformité au cahier des charges Toutes les fonctionnalités requises par vos clients doivent être implémentées. Ce sont vos clients qui évalueront cela.

Style Le code doit être bien structuré (en packages, classes, méthodes, ...). Le code doit suivre les principes de la *programmation orientée objet*, en utilisant le mécanisme de typage, héritage, polymorphisme, liaison tardive, interfaces, classes et méthodes abstraites, ... Le code doit suivre un *style de programmation défensive* : afin de réduire les erreurs lors de l'exécution du programme, le programme doit utiliser au maximum les mécanismes de typage (par exemple les types génériques), d'exceptions et d'assertions. Le code doit suivre proprement le pattern d'architecture Model-View-Controller (MVC). Les *design patterns* doivent être utilisés lors de l'implémentation quand il est possible de les utiliser. N'hésitez pas à utiliser des services et des repositories pour mieux structurer votre code. Les *conventions de nommage* doivent être respectées. Du *code dupliqué*, *code commenté* et du *code obsolète* doivent être évités.

Compilation Il doit y avoir un minimum de (voire aucun) problèmes ou warnings lors de la compilation.

Complexité Le code ne doit pas être inutilement complexe. A tout moment, il faut éviter un style procédural avec des méthodes complexes et beaucoup d'instructions conditionnelles. Les méthodes courtes avec des noms bien choisis seront à privilégier.

Lisibilité Le code doit être facile à lire et à comprendre et doit être bien documenté.

Indépendance de la plate-forme Le code produit doit être indépendant du système d'exploitation. Le code produit sera testé sur trois systèmes d'exploitation différents (MacOS X, Linux et Windows).

Absence de plagiat Le code produit ne peut être distribué entre différents groupes.

Les tests unitaires seront évalués selon les critères suivants :

Processus Le processus de test-driven development a-t-il été suivi ?

Couverture Les tests unitaires couvrent-ils toutes les fonctionnalités requises ? Y a-t-il des tests superflus ? Toutes les classes et méthodes importantes sont-elles couvertes par des tests ?

Exactitude Les tests sont-ils corrects ? Tous les tests unitaires fonctionnent-ils sans échec ou erreur ?

Qualité Les tests sont-ils de bonne qualité (pas de tests trop simples, des tests avec un comportement trivial, ...) ?

Automatisation Est-il possible d'exécuter tous les tests unitaires en une seule fois ? Y a-t-il une suite de tests permettant d'exécuter tous les tests unitaires en une seule fois ?

La documentation sera évaluée selon les critères suivants :

JavaDoc La documentation JavaDoc est-elle présente et est-elle suffisamment complète ? Est-elle de bonne qualité ? Les informations qu'on y trouve sont-elles pertinentes et utiles pour quelqu'un qui veut comprendre ou modifier le code ?

Remarque : bien que la note du client soit importante, celle de votre coach sera bien plus importante. Nous mettons donc bien le focus sur la qualité (du processus de développement et du code développé) plutôt que sur la quantité de travail fourni. Il faut bien évidemment qu'il faille une quantité suffisante de fonctionnalités pour que le projet soit considéré comme réussi.

6.1 Outils

Outil de planification Si besoin est, le choix des outils utilisés pour réaliser la planification et des diagrammes **Gantt** et **PERT** est à votre discrétion. De nombreux outils commerciaux ou open source existent (p.e. **Microsoft Project**, **Gnome Planner**, **TaskJuggler**, ...)

Outil de modélisation Si besoin est, comme outil de modélisation **UML**, vous avez le libre choix. Beaucoup d'autres outils commerciaux sont disponibles en version gratuite "Community Edition" sous la forme de stand-alone ou de plugin pour **IntelliJ**. Il existe également plusieurs outils open source de modélisation **UML** sous la forme de stand-alone ou de plugin pour **IntelliJ**.

Outils de développement Les outils suivants doivent être utilisés :

- La version 18 de Java (ou supérieure).
- Un environnement de développement intégré. Nous vous conseillons l'utilisation d'**IntelliJ 2025.3** (montré pendant les TP préparatoires au projet). Nous ne pourrions pas vous fournir de support en cas d'utilisation d'un IDE alternatif.

- Pour vos fenêtres graphiques, vous devez utiliser la librairie graphique **JavaFX**.
- Pour les tests unitaires, l’outil **JUnit** (version 5 ou supérieure) doit être utilisé. Vous pouvez le trouver sur son site officiel² ; il est intégré par défaut dans **IntelliJ**.

Outil de gestion de versions L’utilisation de **Git** est imposée. Chaque groupe recevra un accès à un dépôt ; un groupe n’aura pas accès aux dépôts des autres groupes. Les enseignants auront constamment accès aux dépôts fournis. Utilisez-les intelligemment.

Système d’exploitation Vous pouvez utiliser n’importe quel système d’exploitation pour développer l’application. La seule contrainte est que tous les livrables après la phase d’implémentation (c.-à-d. le code source, les tests unitaires et la documentation) doivent être indépendants de la plateforme choisie.

7 Objectifs du projet

Les seuls textes qui sont fournis sont des “User Stories”, décrivant ce que l’application doit faire selon le point de vue de différents types d’utilisateurs. Cette description est volontairement lacunaire sur les objectifs à réaliser. Les étudiants doivent raffiner ces “User Stories”, en interagissant avec les assistants qui, en tant que « clients » sont demandeurs de l’application.

Les clients pourront éclaircir certains points restés ambigus ou incomplets. Sur cette base, chaque groupe doit établir un cahier des charges plus complet qu’il devra respecter tout au long du développement.

8 La fin du projet

La fin du projet arrive lors de l’échéance de la quatrième itération. Tous les livrables devront être déposés sur votre dépôt pour maximum le lundi 11 mai 2026 à 23h59.

Vous défendez alors votre projet durant la session. Une vidéo de présentation de toutes les fonctionnalités présentes, des fonctionnalités non finalisées (par rapport à ce qui a été prévu) et de l’architecture du projet pourra être demandée avant le jour de la défense.

Lors de cette défense, qui se déroulera en présence de toute notre équipe (assistants et professeur), vous passerez groupe par groupe³. Vous présenterez devant nous et en 10 minutes, toutes les fonctionnalités présentes, les fonctionnalités non finalisées (par rapport à ce qui a été prévu) et l’architecture du projet. Ensuite, nous poserons des questions *individuellement* à chaque membre du groupe (oralement ou via un logiciel sur mesure). Ces questions peuvent porter sur le code, le design, la méthodologie et les tests. Il sera donc nécessaire que chaque membre du groupe connaisse ces quatre aspects du projet pour pouvoir répondre correctement aux questions.

La cote finale sera calculée en fonction de la qualité de tous les livrables et les fonctionnalités réalisées (70%), ainsi qu’en fonction de la cote obtenue lors de la défense finale (pendant la session) (30%). **Attention : si la note de défense est strictement plus petite que 8/20,**

2. <http://www.junit.org/>

3. Les modalités de l’évaluation pourront changer en fonction des précautions sanitaires en vigueur. Le cas échéant, nous vous tiendrons évidemment alors informés le plus tôt possible.

nous oublions la note du projet et la note de défense devient la note finale pour le cours.

Il n'y a pas de seconde session pour les projets. Seul un examen théorique peut être présenté en seconde session et la note obtenue remplacera celle de l'examen oral de la session du Q2 (30%). Les 70% du projet sont repris tels quels.

Les modalités de l'examen⁴ seront envoyées via l'UV en temps voulu.

A Présentation et contenu des histoires

Les histoires devront comporter les champs suivants :

- le numéro de votre groupe ainsi que le numéro de l'itération courante ;
- un numéro identifiant l'histoire ;
- un titre ;
- une description succincte ;
- le numéro d'itération à laquelle l'histoire a été introduite ;
- un niveau de risque exprimé entre 1 et 3 (fort, moyen, faible) ;
- un statut parmi « *complétée* », « *en cours* », « *à faire* », « *mise à jour* » (en précisant l'objet de la mise à jour), « *abandonnée* » (en précisant la raison de l'abandon, par exemple « *scindée en les histoires 5 et 7* », « *à la demande du client* », ...), et « *récupérée* » (en précisant la raison de la récupération).

Il est à noter

- qu'une histoire complétée ne peut être modifiée qu'à l'aide d'une nouvelle histoire ;
- et qu'une histoire « *en cours* » ou « *à faire* » peut faire l'objet d'une mise à jour, au quel cas un double statut doit être indiqué.

4. vidéo, ordre de passage, ...